

Soluciones a los problemas: Competición 1 Problemas matemáticos sencillos



Competición 2 Problemas de búsqueda binaria y voraces

Solve Me First

```
# Funcion que recibe dos enteros y retorna su suma
def solveMeFirst(a, b):
    # Retorna la suma de a y b
    return a + b
# Leer el primer numero
num1 = int(input())
# Leer el segundo numero
num2 = int(input())
# Llamar a la funcion con los dos numeros y guardar el resultado
res = solveMeFirst(num1, num2)
# Imprimir el resultado
print(res)
```

Cálculo de Prefix Sums

```
# Leer n (tamano del arreglo)
n = int(input())
# Leer la lista de enteros
li = list(map(int, input().split()))
# Crear arreglo prefix sums inicializado con ceros
prefix = [0] * n
# El primer elemento es igual al primero del arreglo original
prefix[0] = li[0]
# Calcular las sumas acumuladas desde el indice 1 hasta el final
for i in range(1, n):
    prefix[i] = prefix[i-1] + li[i]
# Imprimir todos los valores del arreglo prefix separados por espacio
print(*prefix)
```

Simple Array Sum

```
#!/bin/python3

import math
import os
import random
import re
```

```
import sys

#
# Complete the 'simpleArraySum' function below.
#
# The function is expected to return an INTEGER.
# The function accepts INTEGER_ARRAY ar as parameter.
#

def simpleArraySum(ar):
    # Write your code here
    return sum(ar)

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    ar_count = int(input().strip())

    ar = list(map(int, input().rstrip().split()))

    result = simpleArraySum(ar)

    fptr.write(str(result) + '\n')

    fptr.close()
```

Reducir arreglo con costo mínimo

```
n = int(input())
li = list(map(int, input().split()))
print(min(li) * (len(li)-1))
```

Compare the Triplets

```
#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'compareTriplets' function below.
#
# The function is expected to return an INTEGER_ARRAY.
# The function accepts following parameters:
# 1. INTEGER_ARRAY a
# 2. INTEGER_ARRAY b
```

```
#

def compareTriplets(a, b):
    # Write your code here
    pa = 0
    pb = 0
    for i in range(len(a)):
        if a[i] > b[i]:
            pa += 1
        elif a[i] < b[i]:
            pb += 1
    return [pa, pb]

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    a = list(map(int, input().rstrip().split()))

    b = list(map(int, input().rstrip().split()))

    result = compareTriplets(a, b)

    fptr.write(' '.join(map(str, result)))
    fptr.write('\n')

    fptr.close()
```

Mini-Max Sum

```
#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'miniMaxSum' function below.
#
# The function accepts INTEGER_ARRAY arr as parameter.
#

def miniMaxSum(arr):
    # Write your code here
    print(sum(arr) - max(arr), sum(arr) - min(arr))

if __name__ == '__main__':
    arr = list(map(int, input().rstrip().split()))
```

```
miniMaxSum(arr)
```

Índice de Equilibrio en un Arreglo

```
n = int(input())
li = list(map(int, input().split()))
tot = sum(li)
acomulado = 0
for i in range(n):
    if acomulado == tot - acomulado - li[i]:
        print(i)
        break
    acomulado += li[i]
else:
    print(-1)
```

Conectar cuerdas con costo mínimo

```
import heapq

heap = []

n = int(input())
li = list(map(int, input().split()))

for e in li:
    heapq.heappush(heap, e)
sol = 0
while len(heap) > 1:
    a = heapq.heappop(heap)
    b = heapq.heappop(heap)
    sol += a + b
    heapq.heappush(heap, a+b)
print(sol)
```

Super Reduced String

```
#!/bin/python3

import math
import os
import random
import re
import sys
```

```

#
# Complete the 'superReducedString' function below.
#
# The function is expected to return a STRING.
# The function accepts STRING s as parameter.
#

def superReducedString(s):
    # Write your code here
    mar = True
    while mar == True:
        mar = False
        s1 = ""
        p = 0
        while p < len(s):
            if p < len(s)-1 and s[p] == s[p+1]:
                p += 2
            else:
                s1 += s[p]
                p += 1
            if len(s1) != len(s):
                mar = True
        s = s1
    if len(s) == 0:
        return "Empty String"
    else:
        return s

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    s = input()

    result = superReducedString(s)

    fptr.write(result + '\n')

    fptr.close()

```

Minimum Absolute Difference in an Array

```

#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'minimumAbsoluteDifference' function below.

```

```
#
# The function is expected to return an INTEGER.
# The function accepts INTEGER_ARRAY arr as parameter.
#

def minimumAbsoluteDifference(arr):
    # Write your code here
    arr.sort()
    sol = abs(arr[0] - arr[1])
    for i in range(1, len(arr)-1):
        sol = min(sol, abs(arr[i] - arr[i+1]))
    return sol

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    n = int(input().strip())

    arr = list(map(int, input().rstrip().split()))

    result = minimumAbsoluteDifference(arr)

    fptr.write(str(result) + '\n')

    fptr.close()
```

Búsqueda binaria directa

```
li = list(map(int, input().split()))
a = int(input())
ini = 0
fin = len(li)-1
while ini <= fin:
    piv = (ini+fin)//2
    if li[piv] == a:
        print("Si")
        break
    elif li[piv] < a:
        ini = piv+1
    else:
        fin = piv-1
else:
    print("No")
```

Maximum Perimeter Triangle

```
#!/bin/python3

import math
```

```

import os
import random
import re
import sys

#
# Complete the 'maximumPerimeterTriangle' function below.
#
# The function is expected to return an INTEGER_ARRAY.
# The function accepts INTEGER_ARRAY sticks as parameter.
#

def maximumPerimeterTriangle(sticks):
    # Write your code here
    sticks.sort()
    sticks = sticks[::-1]
    for i in range(len(sticks) - 2):
        a, b, c = sticks[i], sticks[i+1], sticks[i+2]
        if b + c > a:
            return [c, b, a]

    return [-1]

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    n = int(input().strip())

    sticks = list(map(int, input().rstrip().split()))

    result = maximumPerimeterTriangle(sticks)

    fptr.write(' '.join(map(str, result)))
    fptr.write('\n')

    fptr.close()

```

Marc's Cakewalk

```

#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'marcsCakewalk' function below.
#
# The function is expected to return a LONG_INTEGER.

```

```
# The function accepts INTEGER_ARRAY calorie as parameter.
#

def marcsCakewalk(calorie):
    # Write your code here
    calorie.sort()
    calorie = calorie[::-1]
    sol = 0
    for i in range(len(calorie)):
        sol += calorie[i] * 2 ** i
    return sol

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    n = int(input().strip())

    calorie = list(map(int, input().rstrip().split()))

    result = marcsCakewalk(calorie)

    fptr.write(str(result) + '\n')

    fptr.close()
```

Ice Cream Parlor

```
#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'icecreamParlor' function below.
#
# The function is expected to return an INTEGER_ARRAY.
# The function accepts following parameters:
# 1. INTEGER m
# 2. INTEGER_ARRAY arr
#

def icecreamParlor(m, arr):
    # Write your code here
    diccionario = {}
    for i in range(len(arr)):
        if (m - arr[i]) in diccionario:
            return [diccionario[m - arr[i]]+1, i+1]
```



```
        diccionario[arr[i]] = i

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    t = int(input().strip())

    for t_itr in range(t):
        m = int(input().strip())

        n = int(input().strip())

        arr = list(map(int, input().rstrip().split()))

        result = icecreamParlor(m, arr)

        fptr.write(' '.join(map(str, result)))
        fptr.write('\n')

    fptr.close()
```